

Picking up last lecture's cliffhanger

Our logistic-regression cheese detector reports **97% accuracy**. Ship it?

It predicts “good” for every batch. Cheese is 3% bad, so being right 97% of the time means catching **zero** bad batches. **Accuracy looked great and was useless.**

This deck: the metrics that actually tell you whether a classifier is any good — and how to turn its *scores* into *decisions*.

Outline

Accuracy and its problem

Precision, recall, F1

Threshold dependence and the ROC fix

When ROC fails: PR and lift

Practical: choosing the metric

Threshold tuning

Wrap-up

Setup: the cheese factory running example

Imagine a cheese factory producing 1000 wheels per day. About **3% have a quality problem** - contamination, spoilage, or off-flavor. Bad cheese reaching customers causes recalls, illness, lawsuits.

Binary classification task:

- ▶ **Positive class** (the rare class, $\sim 3\%$): contaminated / bad cheese batch.
- ▶ **Negative class** (the common class, $\sim 97\%$): clean / good cheese batch.

We build a model that scores each batch's risk – most classifiers output a *probability*, not a hard label. (*That score is more useful than the label; we'll see why halfway through this deck.*) We need:

1. A metric to *compare* candidate models.
2. A way to pick a *threshold* that turns scores into decisions.
3. A method that respects the *asymmetric cost* (missing bad cheese is much worse than over-inspecting clean cheese).

This deck walks through what works, what doesn't, and why. The cascade: accuracy → precision/recall → ROC AUC → PR AUC, each one solving the previous one's problem.

Accuracy: the obvious starting point

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{total predictions}}$$

- ▶ Simple to compute, simple to explain to stakeholders.
- ▶ “What fraction of cheese batches did we classify correctly?”
- ▶ Default metric in many tutorials.

```
from sklearn.metrics import accuracy_score  
accuracy_score(y_true, y_pred)
```

Sounds reasonable. Let's see when it stops being useful. *Next frame: we'll break this down into the four count types (TP, FP, FN, TN) that every other metric is built from.*

Predict-first: where accuracy breaks

Your cheese factory has 1000 daily batches. 970 are clean, 30 are contaminated. You ship a model that predicts “clean” for *every single batch* (no actual model - just the constant).

Predict:

- ▶ What's the accuracy of this “model”?
- ▶ What's its recall on contaminated batches?
- ▶ Would you ship it?
- ▶ **Accuracy:** $970/1000 = 97\%$. Looks excellent.
- ▶ **Recall on positives:** 0%. Catches none of the contaminated batches.
- ▶ **Would you ship?** Customers get sick. Lawsuits. Recall costs millions.

The fundamental problem with accuracy: it weights every error equally. When classes are imbalanced, the rare class becomes invisible. *The metric we need must focus on the positive class.*

Precision, recall, F1

Accuracy weights every error equally, so the rare 3% that matters vanishes.

Fix: stop counting all correctness and **focus on the positive class**.

The confusion matrix

All classification metrics derive from four counts. Lay them on a 2×2 :

	Pred. Positive	Pred. Negative
Actual Pos.	TP bad cheese flagged	FN bad cheese missed!
Actual Neg.	FP wasted inspection	TN clean cheese passed

Asymmetry of cost in the cheese factory:

- ▶ **FN (orange, top-right)**: contaminated cheese reaches customers → recall, illness, lawsuit. *The expensive mistake.*
- ▶ **FP (orange, bottom-left)**: clean batch flagged for extra inspection → wasted hour of staff time. *Annoying but cheap.*
- ▶ **TP (red)**: caught a contaminated batch. *The win.*
- ▶ **TN (green)**: clean batch passed normally. *The default.*

Heads-up: sklearn's `confusion_matrix` returns rows/cols as `[TN, FP; FN, TP]` (negative class first) by default.

Pass `labels=[1,0]` to flip into the positive-first layout shown above.

Precision and recall

Recall (also: sensitivity, TPR):

$$\text{Recall} = \frac{TP}{TP + FN}$$

“Of all bad cheese batches, how many did the model catch?” *What fraction of contaminated cheese reaches customers?* 1 – recall.

For the cheese factory: *recall is the safety metric.* Every percentage point of recall translates to fewer sick customers.

Precision (also: PPV):

$$\text{Precision} = \frac{TP}{TP + FP}$$

“Of all batches the model flagged, how many were actually bad?” *What fraction of inspections were worthwhile?*

For the cheese factory: *precision is the efficiency metric.* Low precision = inspectors chasing false alarms.

```
from sklearn.metrics import precision_score, recall_score
precision_score(y_true, y_pred)    # of flagged, how many real
recall_score(y_true, y_pred)      # of real, how many caught
```

The tension: flagging more batches raises recall but tanks precision. Flagging fewer raises precision but tanks recall.

F1: when you need a single number

Sometimes you need one metric instead of two. **F1 score** is the harmonic mean of precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why harmonic, not arithmetic? Harmonic mean is dominated by the smaller value:

Precision	Recall	Arithmetic mean	Harmonic mean (F1)
1.0	0.0	0.5	0.0
0.8	0.8	0.8	0.8
0.5	0.9	0.7	0.64
0.9	0.5	0.7	0.64

Arithmetic mean lets a 1.0 in one column hide a 0.0 in the other. Harmonic mean refuses. F1 catches the “cheating with extreme threshold” that single metrics permit.

```
from sklearn.metrics import f1_score, fbeta_score
f1_score(y_true, y_pred)
fbeta_score(y_true, y_pred, beta=2)    # recall-leaning
```

F_β generalization: $F_\beta = (1 + \beta^2) \cdot \frac{P \cdot R}{\beta^2 \cdot P + R}$. Higher β weights recall more; $\beta = 2$ (F_2) fits the cheese factory better than F_1 because missing bad cheese is much worse than over-flagging clean.

Worked example: cheese batch inspection

50 cheese batches inspected. 10 actually bad. Model flags 13.

	Pred Bad	Pred Good	Total
Actual Bad (10)	TP = 8	FN = 2	10
Actual Good (40)	FP = 5	TN = 35	40
Total	13	37	50

Metric	Calculation	Value
Accuracy	$(8 + 35)/50$	86%
Recall	$8/(8 + 2)$	80%
Precision	$8/(8 + 5)$	62%
F1	$2 \cdot 0.62 \cdot 0.80 / (0.62 + 0.80)$	70%

What each metric says:

- ▶ Accuracy: "86%, looks good." Misleading - hides the 2 bad batches that got through.
- ▶ Recall: "Caught 80% of contaminated batches." Operationally useful.
- ▶ Precision: "Only 62% of flagged batches were actually bad." 38% wasted inspection effort.
- ▶ F1: "70%" - a single number that punishes the imbalance between P and R.

Thresholds and the ROC curve

Precision, recall and F1 all shift as you move the cutoff c . Fix: summarize a model across *every* threshold at once.

Where precision and recall come from: thresholds

Remember the precision-recall tension from the previous frame? “Flag more $\rightarrow$ recall up, precision down.” **That tension IS the threshold knob.**

Most classifiers don't output labels - they output *scores* or *probabilities*:

$$\text{predict positive} \iff \text{score}(\mathbf{x}) > c$$

The **threshold** c is a knob we choose. Different $c \Rightarrow$ different $(TP, FP, FN, TN) \Rightarrow$ different P/R.

```
scores = model.predict_proba(X)[: , 1]      # probability of positive class
y_pred = (scores > 0.5).astype(int)         # default threshold
```

- ▶ **Lower threshold** ($c \rightarrow 0$): flag more batches as bad $\rightarrow$ catch more (higher recall) but include more false alarms (lower precision).
- ▶ **Higher threshold** ($c \rightarrow 1$): flag fewer batches $\rightarrow$ fewer false alarms (higher precision) but miss more bad cheese (lower recall).
- ▶ **Default** $c = 0.5$ is just one point on a curve. Often not the right point.

Predict-first: P and R at extreme thresholds

Your model outputs scores. Consider two extreme threshold choices.

Predict: for each threshold, what are P and R?

- ▶ $c = 0.0$: predict positive for *every batch*.
- ▶ $c = 1.0$: predict negative for *every batch*.

Reveal:

- ▶ $c = 0$: every batch predicted positive. TP = all real positives, $FN = 0$. So **recall = 100%**. FP = all real negatives. **Precision = base rate** ($\sim 3\%$ for cheese).
- ▶ $c = 1$: every batch predicted negative. $TP = 0$, FN = all positives. **Recall = 0%**. Precision = $0/0$ - sklearn returns 0 by convention.

The problem with P/R as single metrics: each one can be trivially gamed by an extreme threshold. F1 prevents the gaming but *still depends on which threshold you picked*. We need a metric that summarizes performance *across all thresholds at once*.

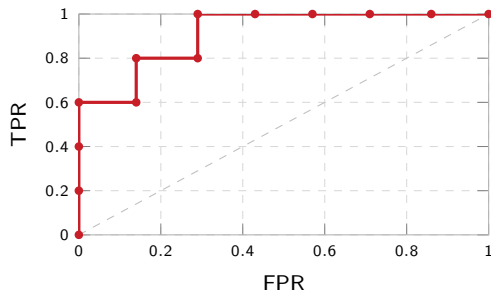
The ROC curve

Two new names for old ratios: **TPR** = recall = $TP/(TP + FN)$ and **FPR** = $FP/(FP + TN)$ – “of all clean batches, what fraction did we false-alarm on?” (also called the false alarm rate).

Sweep the threshold from 1 (no positives) down to 0 (all positives). At each step, record the (FPR, TPR) point. The trail is the **ROC curve**.

Algorithm.

1. Rank predictions by score, highest first.
2. Start at $c = 1 \Rightarrow$ everything predicted negative $\Rightarrow$ (FPR, TPR) = (0,0).
3. Lower threshold so the top-ranked point becomes positive.
4. If real positive: move TPR up by $1/n_+$. If real negative: move FPR right by $1/n_-$.
5. Continue until $c = 0 \Rightarrow$ (FPR, TPR) = (1,1).



ROC curve from 12-batch example. Each step is one ranked prediction crossing the threshold.

Reading the curve: top-left corner = perfect (TPR = 1 at FPR = 0). Diagonal = random guessing

ROC AUC

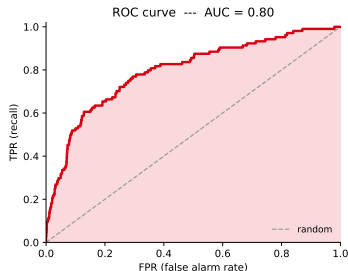
Summarize the whole ROC curve as one number: the **area under the curve**.

- ▶ $AUC = 1.0$: perfect classifier (curve hits top-left).
- ▶ $AUC = 0.5$: random (curve = diagonal).
- ▶ $AUC = 0.0$: perfectly wrong (flip labels for $AUC = 1$).

Probabilistic interpretation. ROC AUC = probability that a random positive scores higher than a random negative. *Equivalent to the Mann-Whitney U / Wilcoxon rank-sum statistic — a real probability, not just an area.*

Threshold-free. You compare models on AUC *without picking a threshold*. The metric summarizes the score's ranking quality.

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_true, y_scores)      # NOT y_pred - pass scores/probabilities!
```



Footgun: pass scores — `predict_proba(X)[: ,1]` — not labels; labels make AUC look like accuracy.

When ROC lies: PR and lift

On rare-positive data the huge negative class hides real differences in ROC.

Fix: measure precision directly — the PR curve.

Predict-first: ROC's blind spot on imbalanced data

Two models on the same imbalanced cheese data: 100 bad batches, 10,000 clean batches.

	TP	FP	FN	TN
Model A	91	10	9	9,990
Model B	91	100	9	9,900

Predict: for each model, compute

- ▶ Recall (TPR)?
- ▶ FPR?
- ▶ Precision (PPV)?
- ▶ Which model would you ship?

	Recall	FPR	Precision
Model A	91%	$10/10,000 = \mathbf{0.10\%}$	$91/101 = \mathbf{90\%}$
Model B	91%	$100/10,000 = \mathbf{1.0\%}$	$91/191 = \mathbf{48\%}$

ROC's blind spot: FPR moved from 0.10% to 1.0% - looks like a 0.9 percentage point difference. *Tiny in ROC space.* Meanwhile, precision crashed from 90% to 48% - half of B's inspections are now wasted. *On imbalanced data, the large negative class dominates the FPR denominator and hides huge operational differences.*

The PR curve

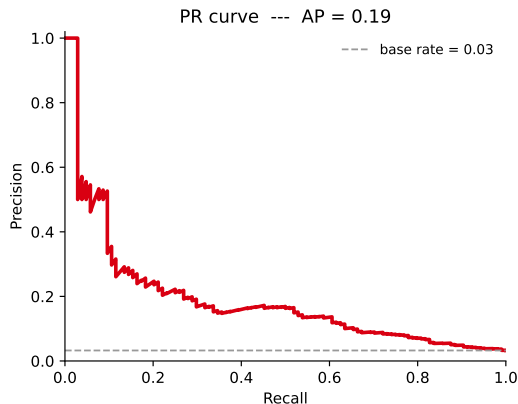
Same sweep over thresholds, but plot **precision vs. recall** instead of TPR vs. FPR.

Reading the PR curve:

- ▶ Top-right corner = perfect (precision = 1, recall = 1).
- ▶ Top-left = high precision, low recall (strict threshold, few but high-quality alarms).
- ▶ Bottom-right = low precision, high recall (loose threshold, many alarms with many false).
- ▶ Baseline: a constant “predict positive” classifier has precision = base rate, recall = 1.

Doesn't depend on TN. The TN count never enters precision or recall. *This is exactly what makes PR informative on imbalanced data.*

Real PR curves zig-zag (precision can rise as recall increases) — visible above; sklearn's `average_precision_score` sums those steps into the AP



Real PR curve (note the zig-zag); dashed baseline = base rate.

PR AUC (Average Precision)

Summarize the PR curve as a single number: **average precision** (the area under the PR curve).

$$AP = \sum_n (R_n - R_{n-1}) \cdot P_n$$

where n indexes the *distinct recall thresholds* produced by sorting predictions by score, and (P_n, R_n) are the precision and recall at threshold n .

- ▶ $AP = 1.0$: perfect ranking.
- ▶ $AP = \text{base rate}$: random ranking. *Not 0.5!*
- ▶ Higher AP = better ranking, weighted toward high-precision regions.

```
from sklearn.metrics import average_precision_score
average_precision_score(y_true, y_scores)
```

Sensitive to imbalance, by design. The model A vs. B example: ROC AUC barely changes; PR AUC drops dramatically because precision drops dramatically. *For imbalanced classification, PR AUC is the metric that matches operational reality.*

Lift and gain charts: the “top- $k\%$ ” question

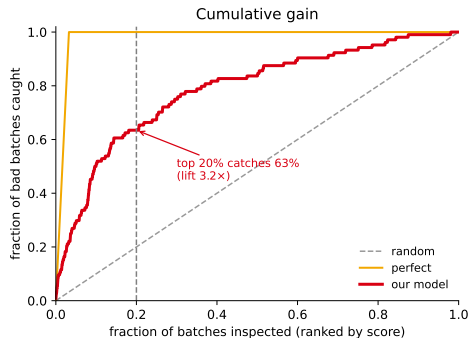
Operational constraint: the cheese factory has inspection capacity for the top 20% of suspicious batches per day. *Which batches should we inspect, and how much better than random?*

Cumulative gain. Rank batches by score (highest first).
x-axis: fraction inspected. y-axis: fraction of bad batches caught. Random = diagonal; perfect = vertical climb to 100% by the base rate.

Lift. $\text{lift}_k = \frac{\% \text{ bad caught in top } k\%}{k\%}$

Random = horizontal line at 1.0. Lift of 5 at the top decile = the model catches 5x more contaminated batches than random at the same inspection budget.

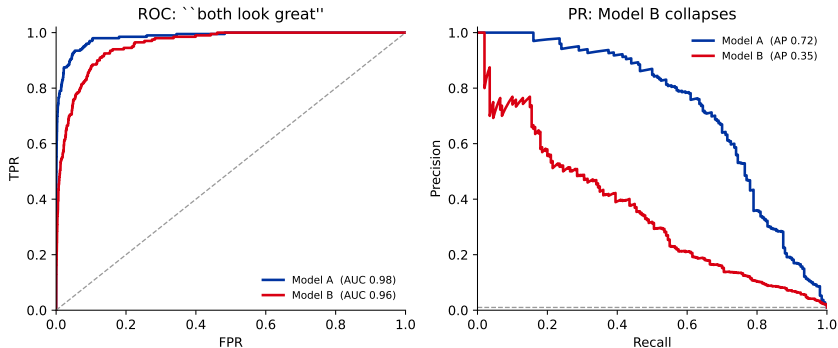
Why this matters. ROC and PR AUC summarize the whole curve; lift answers the question that production actually asks: *given my fixed budget, how much better than random?*



Real cheese model; the annotation reads off the lift at the top 20%.

ROC vs. PR side-by-side on imbalanced data

Same imbalanced dataset (1% positive rate), same two models. Two views:



Same models, two stories. ROC: both near top-left (AUC 0.96–0.98). PR: A holds precision (AP 0.72), B collapses toward baseline (AP 0.35). On imbalanced data, *ROC overstates and PR clarifies* — reach for PR when the positive class is rare.

Which metric for which problem?

Situation	Use	Why
Balanced classes, single threshold	F1 (or accuracy)	Single number; symmetric.
Balanced classes, threshold-free	ROC AUC	Compares models on ranking quality.
Imbalanced + threshold-free	PR AUC (average precision)	ROC overstates; PR matches operational reality.
Bounded inspection budget	Lift at top- $k\%$, gain chart	Capacity is the real constraint.
FN much worse than FP (cheese, fraud, medical screening)	Recall, with precision constraint	Asymmetric cost.
FP much worse than FN (spam, churn intervention)	Precision, with recall constraint	Asymmetric cost, other direction.
Single imbalance-robust number	MCC (Matthews CC)	Symmetric; $\in [-1, 1]$; 0 = random.

Default: report 3-4 metrics. One threshold-free (ROC or PR) plus precision/recall at a chosen threshold.

Calibration (Brier, log loss, ECE) – separate deck.

Multi-class metrics: macro vs. micro vs. weighted

With $K > 2$ classes, binary precision/recall don't apply directly. Compute one-vs-rest per class, then average. Three strategies:

- ▶ **Macro:** average per-class metric, *equal weight per class*. Best when all classes matter equally regardless of size. “How well does the model do on each cheese type?”
- ▶ **Micro:** pool all TP/FP/FN counts across classes, then compute. *Equivalent to accuracy* for single-label classification. Best when overall correctness matters.
- ▶ **Weighted:** per-class metric weighted by class support (number of samples). Reflects the data distribution. *Common default in sklearn reports*.

```
from sklearn.metrics import f1_score, classification_report
f1_score(y_true, y_pred, average="macro")      # fair to small classes
f1_score(y_true, y_pred, average="micro")      # ~ accuracy
f1_score(y_true, y_pred, average="weighted")   # support-weighted

print(classification_report(y_true, y_pred))    # all three + per-class
```

For multi-class ROC AUC: use `roc_auc_score(y, scores, multi_class="ovr", average="macro")`. Macro-OVR is the standard for “how well does the model rank each class?”.

Macro caveat: when some classes have few samples (say < 30), per-class F1 is noisy and macro F1 inherits that noise. Always report support (number of samples per class) alongside.

Picking the operating point

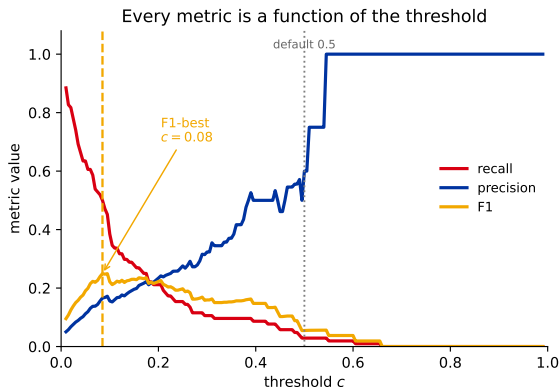
AUC and PR rank models *without* a threshold. To actually deploy, you must choose one cutoff c — and on imbalanced, asymmetric-cost problems 0.5 is almost never it.

Every metric is a function of the threshold

The model outputs scores; the **cutoff** c turns them into decisions. Sweep it:

- ▶ low $c \Rightarrow$ flag more $\Rightarrow$ recall up, precision down;
- ▶ high $c \Rightarrow$ flag fewer $\Rightarrow$ precision up, recall down.

The **default 0.5 is just one point**. Here the F1-best cutoff is ≈ 0.08 — the 3% imbalance pushes it far left of 0.5.



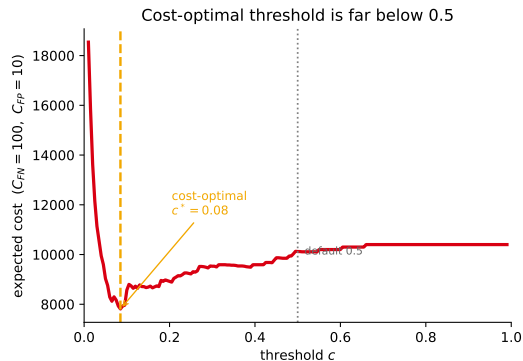
Cost-sensitive thresholding

Choose c to minimize *expected cost*, not a symmetric metric:

$$c^* = \arg \min_c C_{FN} FN(c) + C_{FP} FP(c).$$

With a miss $10\times$ a false alarm, $c^* \approx 0.08$ here — far below 0.5 (real cheese ratios are larger, pushing it lower still).

Tune on a validation set, never on the test set you report — otherwise the chosen threshold is optimistically biased too.



Tuning the threshold in sklearn

TunedThresholdClassifierCV (sklearn ≥ 1.5) post-tunes the cutoff by cross-validation — here to minimize our business cost:

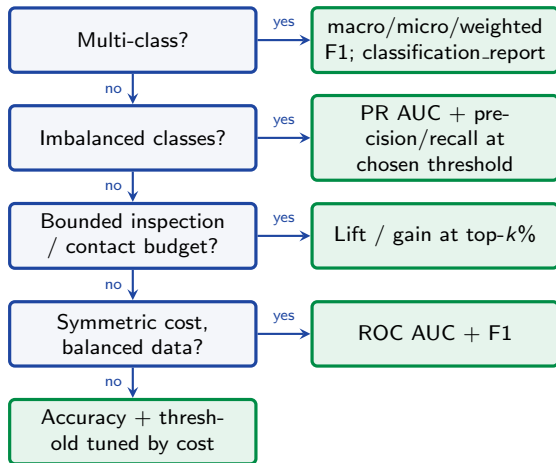
```
from sklearn.model_selection import TunedThresholdClassifierCV
from sklearn.metrics import make_scorer, confusion_matrix

def neg_cost(y, y_pred):                                # negate so "higher = better"
    tn, fp, fn, tp = confusion_matrix(y, y_pred).ravel()
    return -(100 * fn + 10 * fp)

tuned = TunedThresholdClassifierCV(clf, scoring=make_scorer(neg_cost), cv=5)
tuned.fit(X_train, y_train)                             # picks c* by cross-validation
tuned.predict(X_test)                                   # applies the tuned threshold
```

Default scoring maximizes balanced accuracy; pass any scorer (or a business cost) to match your problem. *Fit it on train/validation, never on test.*

Decision flowchart



Always report multiple metrics. A single number hides the trade-off; the trade-off is the story.

Caveat: real problems often hit multiple branches at once (multi-class AND imbalanced, bounded budget AND asymmetric cost). The tree picks a *primary* metric; report secondaries too.

Recap: the cascade

The metric cascade – each one fixes the previous one's problem:

1. **Accuracy** - simple, fails on imbalanced data (97% accuracy with 0% recall).
2. **Precision and recall** - focus on the positive class. *But depend on the threshold.*
3. **F1** - one number that punishes asymmetric P/R. *Still depends on the threshold.*
4. **ROC AUC** - threshold-free; ranking quality. *Overstates on imbalanced data.*
5. **PR AUC (average precision)** - threshold-free AND sensitive to imbalance. The right metric for rare-class problems.
6. **Lift / gain** - the operational “top- $k\%$ ” question when capacity is bounded.

Practical machinery – not part of the cascade, but you'll need them:

- ▶ **Multi-class:** macro for fairness across classes, micro for overall accuracy, weighted for distribution-aware.
- ▶ **Threshold tuning:** given the cost asymmetry, the optimal threshold rarely equals 0.5.

One-line takeaway: report multiple metrics, always include one that matches your data's imbalance and your problem's cost structure.